Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Uvod u digitalnu obradu slike
Predmetni profesor: prof. dr Emir Sokić
Autor: Berina Biberović
Akademska godina: 2022/2023.

Tutorijal 2

Pikseli unutar slike. Prostori boja.

Manipulacija tačkama/koordinatama

Važan dio obrade slike je mogućnost pristupanja pojedinačnim pikselima unutar učitane slike. Za te svrhe unutar OpenCV-a postoje posebne klase koje nam omogućavaju upravo to.

`template<typename _Tp> class cv::Point_<_Tp>` klasa

Klasa `template<typename _Tp> class cv::Point_<_Tp>` je šablonska klasa za 2D tačke koja je opisana svojim koordinatama x i y. Ovdje, kao što se može pretpostaviti, varijabla unutar šablona `_Tp` može biti tipa `int`, `double`, `float`. Dakle, ukoliko npr. želimo napraviti instancu ove klase gdje su koordinate tipa `int`, onda možemo pisati:

```
cv::Point_<int> tacka(3, 5);
```

Ova klasa se nalazi unutar `core` modula, preciznije unutar `core/types.hpp`. Bitno je istaći da postoji cast operator koji konvertuje automatski koordinate u specificirani tip. Konverzija iz decimalnih tipova u cjelobrojne se radi preko zaokruživanja. Isto tako, postoje još definirane operacije koje su dozvoljene da se rade nad `template<typename _Tp> class cv::Point_<_Tp>` tipovima, koje možete naći na ovom linku (kao i još neke detalje o `template<typename _Tp> class cv::Point_<_Tp>` klasi).

`template<typename _Tp> class cv::Point3_<_Tp>` klasa

Slično kao prethodno spomenuta klasa, `template<typename _Tp> class cv::Point3_<_Tp>` klasa je šablonska klasa koja je specificirana sa tri koordinate x, y i z. Kao i u prethodnoj klasi, koordinate 3D tačaka se mogu konvertovati u drugi tip. Vektorska aritmetika i operacije upoređivanja su podržane za ovu klasu. Na sličan način se incijaliziraju instance ove klase kao i prethodne, samo što je potrebno proslijediti tri parametra umjesto dva. Detalje možete pronaći na ovom linku.

Pristupanje pikselima unutar objekata klase `cv::Mat`

Da bismo mogli pristupiti vrijednosti piksela unutar slike moramo znati tip slike i broj kanala. Postoji više tipova slike, kao što su binarne, *grayscale*, indeksirane, RGB, itd. Ovaj tutorijal će preći kako pristupiti pikeslima jednokanalne *grayscale* slike i RGB trokanalne slike.

Da bi se pristupilo pikselu sa koordinatama x i y unutar jednokanalne *grayscale* slike, potrebno je napisati:

```
cv::Scalar intenzitet = slika.at<uchar>(y, x);
```

Klasa `cv::Scalar` je šablonska klasa (`template<typename _Tp> class Scalar_<double>`) koja se nalazi unutar `core/types.hpp` modula. Ova klasa je derivirana iz klase `cv::Vec` (nećemo se baviti detaljima `cv::Vec` klase) i predstavlja 4-elementni vektor. Instance ove klase se mogu koristiti

kao tipični 4-elementni vektori i mogu se konvertovati iz i u `cv::CvScalar` tip. Tip `cv::Scalar` se široko koristi u OpenCV-u kako bi se vrijednosti piksela mogli čitati i pisati.

Tip `uchar` je samo `typedef unsigned char uchar` (dakle, vrijednost intenziteta piksela ne može biti negativna u ovom kontekstu!).

Unutar C++ jezika `intenzitet.val[0]` sadrži vrijednost od 0 do 255. Potrebno je obratiti pažnju na redoslijed koordinata x i y - kako su u OpenCV-u slike predstavljene kao ista struktura kao što su matrice, onda se ista konvencija koristi za oba slučaja: y koordinata predstavlja red matrice, dok x koordinata predstavlja kolonu matrice. Alternativno, može se pisati i:

```
cv::Scalar intenzitet = slika.at<uchar>(cv::Point(x, y));
```

Da bismo pristupili pikselu sa x i y koordinatama unutar trokanalne BGR slike (podrazumijevani format koji se učitava svaki put kada koristimo `cv::imread`) trebamo pisati:

```
cv::Vec3b intenzitet = slika.at<cv::Vec3b>(cv::Point(x, y));
uchar plava = intenzitet.val[0];
uchar zelena = intenzitet.val[1];
uchar crvena = intenzitet.val[2];
```

Klasa `cv::Vec3b` se nalazi unutar modula `core/matx.hpp` i nasljđena je iz prethodno spomenute klase `cv::Vec`, koja u ovom slučaju predstavlja 3-elementni vektor čiji su elementi tipa `uchar`. Alternativno, može se ista metoda iskoristiti za *floating point* slike (koje se mogu dobiti npr. kada se Sobel transformacija uradi nad trokanalnom slikom) ako se umjesto `cv::Vec3b` u prethodnom isječku koda piše `cv::Vec3f`.

U prethodnom isječku koda, svaka od vrijednosti `vrijednost.val[0]`, `vrijednost.val[1]` i `vrijednost.val[2]` mogu biti od 0 do 255. Primjetimo da, iako u govoru ove slike nazivamo RGB slikama, redoslijed unutar OpenCV-a nije RGB, nego BGR (veoma važno je paziti na taj redoslijed u toku testova, jer mnoge greške se naprave tu).

Postoje još neke metode koje se mogu raditi nad pikselima slika i različiti načini na koji se oni mogu predstavljati, ali u ovom tutorijalu nećemo više obrađivati tu temu, nego u budućim tutorijalima ako bude bilo potrebe za tim.

Prostori boja

Prostori boja (eng. *color spaces*) predstavljaju specifičan način organizacije boja. Neki prostor boja podržava reprodukcibilne reprezentacije boja - bile one u analgonom ili digitalnom formatu. Model boje (eng. *color model*) je apstraktni matematički model koji opisuje način na koji boje mogu biti predstavljene kao n-torke brojeva. Ukoliko dodamo specifičnu funkciju mapiranja između modela boje i referentnog prostora boje postižemo unutar referentnog prostora boje definitivni "otisak" (tzv. gamut) i za dati model boje ovo definira prostor boje. Kako prostor boja inače znači određenu kombinaciju modela boje i funkcije mapiranja, prostor boje se inače koristi kao sinonim za model boje, ali ovo korištenje je netačno u strogom obliku. Npr. iako postoje više specifičnih prostora boja koji se zasnivaju na RGB modelu boje, ne postoji singularni RGB prostor boje.

Konverzije prostora boja

Na predavanju 2 je bilo spomenuto nekoliko prostora boja kao što su npr. RGB, CMY i CMYK, Lab, HSV, HSL, itd. Pored toga, na predavanju 2 su navedene funkcije konverzije iz jednog prostora boja u drugi. Unutar C++ mi možemo definirati naše funkcije koje bi radile konverzije onako kako su one navedene na predavanju, ali, srećom OpenCV nam omogućava da ne moramo te konverzije raditi "ručno", već postoje funkcije koje automatski rade konverziju za nas.

Unutar OpenCV-a, da bismo konvertovali sliku iz jednog u drugi prostor boje, možemo koristiti funkciju `cv::cvtColor(InputArray izvorna_slika, OutputArray destinacijska_slika, int kod,`

`int br_kan = 0)`: parametar `izvorna_slika` predstavlja sliku (8-bitna unsigned, 16-bitna unsigned ili single-precision floating point) koju želimo prebaciti u novi prostor boja; parametar `destinacijska_slika` je rezultantna slika obrade konverzije koja mora biti iste veličine i dubina kao `izvorna_slika`; `kod` predstavlja konverzijski kod prostora boja (listu možete naći ovdje); a `br_kan` predstavlja broj kanala u rezultantnoj slici - ukoliko je taj parametar jednak 0, onda broj kanala se automatski očitava iz `izvorna_slika` i `kod`. Ova funkcija se nalazi unutar modula `imgproc.hpp`, tako da je potrebno taj modul uključiti na početku programa.

Slijedi jednostavan primjer učitavanja BGR slike, prebacivanja je u *grayscale*, HSV i binarnu sliku.

```
#include <opencv4/opencv2/core/mat.hpp>
#include <opencv4/opencv2/highgui.hpp>
#include <opencv4/opencv2/imgcodecs.hpp>
#include <opencv4/opencv2/imgproc.hpp>

int main() {

    // Ucitavanje BGR slike:
    cv::Mat BGR_slika =
        cv::imread("/home/anniberi/Pictures/Wallpapers/cat.jpeg",
        cv::IMREAD_COLOR);

    // Deklarisanje grayscale, binarne i HSV slike:
    cv::Mat gray_slika, bin_slika, HSV_slika;

    // Koristenje cv::cvtColor funkcije kako bismo dobili grayscale i HSV
        sliku:
    cv::cvtColor(BGR_slika, gray_slika, cv::COLOR_BGR2GRAY);
    cv::cvtColor(BGR_slika, HSV_slika, cv::COLOR_BGR2HSV);

    // Koristenje threshold funkcije kako bismo dobili binarnu sliku:
    cv::threshold(gray_slika, bin_slika, 120, 255, cv::THRESH_BINARY);

    // Prikaz svih slika:
    cv::imshow("BGR slika", BGR_slika);
    cv::imshow("Grayscale slika", gray_slika);
    cv::imshow("HSV slika", HSV_slika);
    cv::imshow("Binarna slika", bin_slika);

    cv::waitKey();

    return 0;
}
```

Možemo primijetiti da, kako bismo sliku prebacili iz *grayscale* u binarnu nismo koristili funkciju `cv::cvtColor` nego smo koristili funkciju `cv::threshold`. Ova funkcija prima 5 parametara: ulaznu *grayscale sliku*; izlaznu sliku koja će biti rezultat pretvaranja ulazne slike u binarnu sliku; parametar tip `double` koji predstavlja prag intenziteta piksela gdje sve manje od tog praga predstavlja 0, a sve veće od njega predstavlja 1; parametar tipa `double` koji predstavlja maksimalnu vrijednost koja se koristi sa *thresholding* `enum` tipovima `cv::THRESH_BINARY` i `cv::THRESH_BINARY_INV`; i na kraju parametar tipa `int` koji predstavlja upravo `enum` *thresholding* tip.

Još jedan tip *thresholding*-a koji postoji je adaptivni thresholding kojeg ćemo pokazati na jednostavnom primjeru.

```
#include <opencv4/opencv2/core/mat.hpp>
#include <opencv4/opencv2/imgproc.hpp>
#include <opencv4/opencv2/imgcodecs.hpp>
#include <opencv4/opencv2/highgui.hpp>
```

```
5
6  int main() {
7
8      // Ucitavnanje slike u grayscale-u:
9      cv::Mat slika =
           cv::imread("/home/anniberi/Pictures/Wallpapers/cat.jpeg",
           cv::IMREAD_GRAYSCALE);
10     cv::Mat binarna;
11
12     // Pravljenje trackbar-ova:
13     cv::namedWindow("Trackbars");
14     cv::createTrackbar("Threshold 1", "Trackbars", nullptr, 255);
15     cv::createTrackbar("(Block size - 3) / 2", "Trackbars", nullptr, 255);
16     cv::namedWindow("Adaptivni thresholding");
17     cv::namedWindow("Originalna grayscale slika");
18
19     int t1, t2;
20
21     do {
22
23         // Ucitavanje t1 i t2 sa trackbarova:
24         t1 = cv::getTrackbarPos("Threshold 1", "Trackbars");
25         t2 = cv::getTrackbarPos("(Block size - 3) / 2", "Trackbars");
26
27         // Sada primjenjujemo funkciju za adaptivni thresholding:
28         cv::adaptiveThreshold(slika, binarna, 255,
               cv::ADAPTIVE_THRESH_MEAN_C, cv::THRESH_BINARY, 2 * t2 + 3, t1);
29         // Prikaz slika:
30         cv::imshow("Originalna grayscale slika", slika);
31         cv::imshow("Adaptivni thresholding", binarna);
32
33     }while(cv::waitKey() != 'q');
34
35     cv::destroyAllWindows();
36
37     return 0;
38 }
```

U gornjem kodu smo iskoristili da originalnu sliku odmah učitamo u *grayscale* tipu, koristeći parametar `cv::IMREAD_GRAYSCALE` unutar funkcije `cv::imread`. Nakon toga smo korištenjem funkcije `cv::namedWindow` napravili prozor pod imenom "Trackbars" u kojem ćemo staviti *trackbar*-ove koji će nam koristiti za adaptivni *thresholding*. Funkcija `cv::createTrackbar` prima 6 parametara: prvi parametar je tipa `const String &` i predstavlja ime koje dajemo specifičnom *trackbar*-u; drugi parametar je isto tipa `const String &` i predstavlja ime prozora kojeg smo kreirali u prethodnoj liniji koda; treći parametar je tipa `int*` i predstavlja pokazivač na cjelobrojnu varijablu čija vrijednost reflektuje poziciju *slider*-a i prilikom kreiranja pozicija *slider*-a se definira preko ove varijable (u našem slučaju je to `nullptr` tako da ne pokazuje ni na šta na početku, ali to je ok jer mi možemo pomjerati *slider* lijevo-desno); četvrti parametar je tipa `int` i predstavlja maksimalnu poziciju do koje može ići *slider* - kod nas je to 255; zadnja dva parametra su predefinirana i predstavljaju tipove `cv::TrackbarCallback` i `void *` koji su postavljeni automatski na 0 pri pozivanju funkcije.

Nakon što smo napravili dva ista *trackbar*-a sa različitim imenima unutar istog prozora, pravimo još dva prozora - za *grayscale* sliku i za sliku koju mijenjamo adaptivnim *thresholding*-om. Deklarišemo onda dvije varijable koje će nam služiti za uzimanje trenutne vrijednosti oba *trackbar*-a - `t1` i `t2`.

Unutar `do while` petlje učitavamo vrijednosti u `t1` i `t2` preko funkcije `cv::getTrackbarPos` koja prima dva parametra - ime *trackbar*-a kojeg želimo pročitati i ime prozora u kojem se taj

trackbar nalazi.

Onda primjenjujemo funkciju `cv::adaptiveThreshold` koja prima 7 parametara: prvi parametar je ulazna *grayscale* slika; drugi parametar je izlazna binarna slika; treći parametar je tipa `double` i predstavlja nenultu vrijednost koja se daje pikselima za koje je uslov ispunjen; četvrti parametar je tipa `int` i predstavlja koji algoritam se treba koristiti za adaptivni *thresholding*; peti parametar je tipa `int` i predstavlja *thresholding* tip koji mora biti ili `cv::THERSH_BINARY` ili `cv::THRESH_BINARY_INV`; šesti parametar je isto tipa `int` i predstavlja veličinu susjedstva piksela koja se koristi da bi se izračunala *threshold* vrijednost za piksel; sedmi parametar je tipa `double` i predstavlja konstantu koja se oduzima od srednje vrijednosti. Više detalja možete pronaći na linku.

Nakon toga unutar odgovarajućih prozora prikazujemo *grayscale* sliku i sliku koju smo dobili kroz *thresholding*. Na slici 1 je prikazana originalna *grayscale* slika, dok je na slici 2 prikazana slika nakon primjenjivanja adaptivnog *thresholding*-a. Studentima se savjetuje da prođu kroz sve primjere sa tutorijala i sami probaju ubaciti neke svoje slike u program i vide šta se dešava sa slikama koje su na ovaj način promijenili.



Slika 1: Originalna *grayscale* slika



Slika 2: Binarna slika nakon *thresholding*-a

Napomene

Korištena tema za prikaz C++ koda na tutorijalima je `solarized`[1].

Literatura

[1] Ethan Schoonover. *Solarized*. `http://ethanschoonover.com/solarized`. Pristup stranici: 20. maj 2023. 2011.